

## Relatório lista 2 estruturas de dados

O sistema foi implementado utilizando três listas encadeadas simples: uma para armazenar as ordens de compra, outra para as ordens de venda e uma terceira para registrar as transações executadas. Cada nó dessas listas contém a estrutura correspondente, *order* ou *transaction*, e um ponteiro para o próximo elemento. A escolha por listas encadeadas foi feita visando simplicidade e flexibilidade, uma vez que essa estrutura permite inserções e remoções dinâmicas sem necessidade de realocação ou deslocamento de elementos, ao contrário de estruturas baseadas em arrays.

Quanto aos trade-offs considerados, optou-se por manter as listas de ordens não ordenadas. Tal decisão simplificou a implementação da inserção, mas implica um custo adicional em outras operações, como de buscas em geral, já que é necessário percorrer toda a lista correspondente (de compra ou venda) para encontrar a melhor ordem compatível.

Outro ponto relevante foi a forma de remoção das ordens. A operação de cancelamento e a remoção após execução de uma transação são feitas percorrendo a lista até encontrar o elemento desejado, o que também possui custo linear. A busca por contraparte foi implementada percorrendo toda a lista oposta à ordem inserida. Para ordens de compra, busca-se a ordem de venda com menor preço, e, em caso de empate, a de menor timestamp. Para ordens de venda, busca-se a ordem de compra com maior preço, também com desempate por timestamp. Por fim, os gets possuem também ordem geral linear, pois exigem não só percorrer toda a lista encadeada, como copiar seus elementos um a um. Dessa forma, observa-se que o desempenho do sistema em geral é linear em relação ao número de ordens armazenadas. Embora essa abordagem seja suficiente para os requisitos do projeto, o desempenho pode se degradar em cenários com grande volume de dados, sendo possível utilizar estruturas mais sofisticadas para otimização em aplicações reais.